

Name: Student ID:

ECS401: Unit 11 Lab Progress Test

These exercises are to help you judge your progress. It is really important that you make sure you fully understand everything we have covered so far.

1. Write a statement that links a variable called `haikuFile` to a file called `haiku.txt` to be used to read a Haiku poem in to a program from the file.

Answer:

2. Write a line of code that reads a single line of text from a variable called `haikuFile` into a String variable called `line1`.

Answer:

3. Write a line of code to close the file `haikuFile`.

Answer:

4. What needs to be added to the end of the header line of any method doing file input or output (including all methods in the call stack back to main that leads to it being called)?

Answer:

5. Write a statement that links a variable called `haikuFile` to a file called `haiku.txt` to be used to write a Haiku poem out from a program to the file.

Answer:

6. Write a for loop that outputs the THREE lines of a Haiku to a file via a variable called `haikuFile` linked to the file for output. Assume the lines of the Haiku are stored in a String array variable called `myHaiku`.

Answer:

7. Explain briefly what would happen and why, if a program tried to write to a file that was still open for writing by another program if using the standard I/O mechanism as used in the lectures.



Answer:

8. Which of the following fragments of code (a) - (e) below could be used to read in all the data from a file into an array of Student records. The file is stored in the format of one piece of data per line consisting of the student name followed by the mark they received on the next line, followed by the next name and next mark, and so on. The Student record is defined as follows:

```
class Student
{
    int mark;
    String name;
}
```

The number of student records stored in the file is stored in variable ENTRIES. You can assume in each case there is appropriate other code to set up and close inputStream.

a)

```
for(int i = 0; i < ENTRIES; i++)
{
    String name = inputStream.readLine();
    int mark = Integer.parseInt(inputStream.readLine());
}
```

b)

```
Student [] student = new Student[ENTRIES];
String [] name = new String[ENTRIES];
int [] mark = new int[ENTRIES];

for(int i = 0; i < ENTRIES; i++)
```

```
{
    name[i] = inputStream.readLine();
    mark [i] = Integer.parseInt(inputStream.readLine());
}
```

c)

```
Student [] student = new Student[ENTRIES];

for(int i = 0; i< ENTRIES; i++)
{
    student[i].mark = Integer.parseInt(inputStream.readLine());
    student[i].name = inputStream.readLine();
}
```

d)

```
Student [] student = new Student[ENTRIES];

for(int i = 0; i< ENTRIES; i++)
{
    student[i].name = inputStream.readLine();
    student[i].mark = Integer.parseInt(inputStream.readLine());
}
```

e)

```
Student [] student = new Student[ENTRIES];

for(int i = 0; i< ENTRIES; i++)
{
    student.name[i] = inputStream.readLine();
    student.mark[i] = Integer.parseInt(inputStream.readLine());
}
```

Answer: